1. Linear search: trace and explain

TEACHER KEY - teaching examples, not a secure test

WAGBA: Follow and explain search algorithms so we can choose the correct steps to find an item.

Start with Questions 1-5. Mr Dave will tell you which follow-up questions to do. Question 11 is an optional challenge.

The librarian needs to find **target 13**. Follow the supplied linear-search algorithm in order. Do not jump straight to the answer you can see.

Index	0	1	2	3	4	5	6
Stored value	**5**	**9**	**13**	**18**	**24**	**31**	**42**

Algorithm reference: line numbers help you locate instructions.

```
01  position ← 0
02  found ← False
03  WHILE position < LEN(items) AND found = False
04      IF items[position] = target THEN
05          found ← True
06      ELSE
07          position ← position + 1
08      ENDIF
09  ENDWHILE
10  OUTPUT found
```

Read the symbols: ← stores a value; = compares values. **found** is True or False. The algorithm outputs **found**, not the matching index.

1. Which values will linear search check to find 13? Complete the two blank rows below.

The first row is a worked example. In each row record the index, its stored value, whether it equals 13, and what the algorithm does next.

Index (position)	Stored value	Equals target 13?	Next action
0	5	No	Set position to 1
1	9	No	Set position to 2
2	13	Yes	Set found to True

2. Why does this search stop after finding 13? State the final value of found and what line 10 outputs.

Model answer: 13 matches the target at index 2, so found becomes True. Line 10 outputs True. The WHILE condition includes found = False; this is now false, so the loop does not repeat.

3. A student checks value 5 and immediately reports "not found". Why is this too early, and which index should the search check next?

Model answer: Only index 0 has been checked. The target could be later in the list. The algorithm should advance to index 1 and compare its value, 9, with 13.

2. Binary search: read and trace

TEACHER KEY - teaching examples, not a secure test

WAGBA: Follow and explain search algorithms so we can choose the correct steps to find an item.

Now search for **target 13** using binary search. The list is sorted from smallest to largest. Index labels stay fixed; no items are deleted or renumbered.

Index	0	1	2	3	4	5	6
Stored value	**5**	**9**	**13**	**18**	**24**	**31**	**42**

low, high and mid are indexes (positions). items[mid] is the value stored at the midpoint index. DIV 2 uses whole-number division.

```
low ← 0
high ← LEN(items) − 1
found ← False
WHILE low ≤ high AND found = False
    mid ← (low + high) DIV 2
    IF items[mid] = target THEN
        found ← True
    ELSE
        IF items[mid] < target THEN
            low ← mid + 1
        ELSE
            high ← mid − 1
        ENDIF
    ENDIF
ENDWHILE
OUTPUT found
```

4. Which items does binary search check to find 13? Complete the two blank rows in the trace.

The first row is a worked example. Record low, high and mid **before** changing a boundary. Write the stored value at mid. In "Next action", state the boundary update or set found to True.

low (first index)	high (last index)	mid (index)	Value at items[mid]	Next action
0	6	3	18	Set high to 2; keep indexes 0-2
0	2	1	9	Set low to 2; keep index 2
2	2	2	13	Set found to True; loop then stops

Check before turning over: the index columns contain positions, not book numbers. On the next page you will explain why the boundaries changed.

3. Binary search: explain each decision

TEACHER KEY - teaching examples, not a secure test

WAGBA: Follow and explain search algorithms so we can choose the correct steps to find an item.

Use the same sorted list. Give both a decision and a reason. Complete Question 5 first; Questions 6 and 7 are follow-up practice when your teacher asks.

Index	0	1	2	3	4	5	6
Stored value	**5**	**9**	**13**	**18**	**24**	**31**	**42**

5. For target 13, the first check uses midpoint index 3, storing value 18. What should the new last index (high) be, and why?

Model answer: high becomes 2, not value 13 or value 18. Target 13 is smaller than 18. Sorted order rules out indexes 3-6; the possible positions left are 0-2. Therefore high = mid - 1 = 2.

6. For target 31, binary search checks value 18 at index 3. A student writes low ← mid. Why should index 3 be excluded, and what should the new first index (low) be?

Model answer: 18 is not 31, so the checked index 3 cannot be a match. Because the list is sorted and 31 is larger than 18, indexes 0-3 are ruled out. The next possible index is 4, so low = mid + 1 = 4. Keeping the old midpoint can prevent progress in some searches.

7. For target 20, the algorithm has checked 18, 31 and 24. It now has low = 4 and high = 3 (both indexes). What does this tell you about the possible search positions? State found and the output on line 16.

Model answer: There are no possible positions left: the first index, 4, is beyond the last index, 3. The condition low ≤ high is false, so the loop stops. found remains False and line 16 outputs False. The original list has not changed.

4. Compare the two searches

TEACHER KEY - teaching examples, not a secure test

WAGBA: Follow and explain search algorithms so we can choose the correct steps to find an item.

Follow-up when your teacher asks: search for **target 42** using both algorithms on pages 1 and 2. Both stop after finding a match.

Index	0	1	2	3	4	5	6
Stored value	**5**	**9**	**13**	**18**	**24**	**31**	**42**

Counting rule: one item check means inspecting one stored list value against the target. Do not count each individual operator or WHILE test.

8. Which values does linear search inspect, in order, including the matching value 42? Give the matching index and the total number of item checks.

Model answer: Checked values: 5, 9, 13, 18, 24, 31, 42. Matching index: 6. Item checks: 7.

9. Which positions and values does binary search inspect for target 42? Complete one row for each item check, then write its matching index and total checks.

Record the boundary indexes before updating them, as you did in Question 4.

low (index)	high (index)	mid (index)	Stored value	Next action
0	6	3	18	low = 4
4	6	5	31	low = 6
6	6	6	42	found = True

Matching index: 6. Item checks: 3.

10. Which algorithm uses fewer item checks for target 42? Use both counts and explain why sorted order helps binary search.

Model answer: Binary search uses 3 item checks; linear search uses 7. Sorted order lets binary search rule out a section after each non-matching midpoint comparison instead of inspecting every earlier item.

11 (optional challenge). Does binary search always use fewer item checks? Test that claim with target 5. List the checked values and counts for both searches, then give your conclusion.

Model answer: Linear: checks 5 only (1 item check). Binary: checks 18, 9, 5 (3 item checks). Therefore binary search does not always use fewer item checks for an individual target.

Next step: Which specific step do you need to practise next? Use Question 1-11 as evidence. Complete your full KSU Learning Pit Stop on the website, or on paper if time is called.

Look for an actionable target, such as "distinguish mid from items[mid]" or "update high to mid - 1 and explain why". Do not judge understanding from speed or completion alone.